

Blender Batch Renderer

User guide & technical reference

Version 0.6.0 · October 2026

Queue, render, stop and resume Blender jobs – overnight, unattended.

Contents

1 Overview . . . 5
1.1 Key features . . . 5
1.2 Guiding principles . . . 5
2 Installation & starting . . . 7
2.1 Requirements . . . 7
2.2 One-time setup . . . 7
2.3 Starting the app . . . 7
2.4 Updates without rebuilding . . . 7
2.5 First launch . . . 8
3 The window . . . 9
3.1 Live tab . . . 9
3.2 Job settings tab . . . 10
3.3 Issues tab . . . 10
3.4 Machines tab and mode switch . . . 10
4 Working with the queue . . . 10
4.1 Adding files . . . 10
4.2 Folders . . . 10
4.3 Render order and skipping . . . 11
4.4 Removing jobs . . . 11
4.5 Queue files . . . 11
4.6 Keyboard shortcuts . . . 11
5 Rendering, stopping & resuming . . . 12
5.1 How resuming works . . . 12
5.2 Job states . . . 13
6 Failures, retries & restarts . . . 13
6.1 Frame failures . . . 13
6.2 Files that will not render . . . 13
6.3 Frame timeout . . . 14
6.4 Restart Blender . . . 14
7 Safety checks . . . 14
7.1 Disk space . . . 14

7.2 Changed or missing .blend files . . . 14
7.3 Moved render folders . . . 14
7.4 One copy at a time . . . 14
7.5 Keeping the computer awake . . . 14
8 Blender versions . . . 15
9 Settings reference . . . 15
9.1 Job settings . . . 15
9.2 App settings . . . 16
9.3 What the app changes in your file . . . 17
10 Output folders & data files . . . 17
10.1 frames.csv . . . 17
10.2 report.xlsx . . . 17
10.3 Where the app keeps its own data . . . 18
11 Preview video . . . 18
12 Preflight check . . . 18
13 Status page & API . . . 18
13.1 The status page . . . 18
13.2 On your phone or another computer . . . 19
13.3 Security . . . 19
13.4 API reference (version 1) . . . 19
14 Blender add-on “Send to Batch Renderer” . . . 20
14.1 Installing . . . 20
14.2 Using it . . . 20
15 Rendering on several computers . . . 21
15.1 Setting it up . . . 21
15.2 What happens . . . 22
15.3 Sending the files . . . 22
15.4 Consistency check and the last frames . . . 23
15.5 Stopping, failures and lost machines . . . 23
15.6 Statistics . . . 24
15.7 Security . . . 24
15.8 Where the files are . . . 24
16 How it works . . . 25

16.1 Communication with Blender 25
16.2 Stopping cleanly 26
16.3 Estimates 26
16.4 Source code layout 27
17 Command line 28
18 Troubleshooting & limitations 28
Known limitations 29
Testing 29

1 Overview

Blender Batch Renderer is a desktop app that renders a queue of **.blend** files one after another – typically overnight, with nobody watching. Its central promise: **you can stop, cancel or quit at any moment, and the batch later continues from the last finished frame**. Nothing that was finished is rendered twice, and nothing half-written is ever mistaken for a finished frame.

The app runs Blender in the background (no Blender window), one job at a time, and watches it: it shows live progress, estimates when the batch will finish and how much disk space it will use, retries frames when Blender crashes, records statistics for every frame, and turns each finished job into a quick MP4 preview.

1.1 Key features

- **Resume anywhere** – progress is saved after every frame; stop, quit or crash and press Start again.
- **Several Blender versions in one batch** – each file is matched to the install it was saved with.
- **Folders** to organise the queue, with drag & drop and per-folder on/off switches.
- **Retries** per frame with an optional waiting time, a frame timeout and optional regular Blender restarts.
- **Live view**: current frame and samples, Blender's own remaining-time estimate, ETA for job and batch, projected disk usage, memory, thumbnail of the last frame, charts and Blender's console.
- **Data per job**: CSV files written live, an Excel report with charts, full logs and crash reports.
- **MP4 preview** made automatically when a job finishes (or on demand).
- **Safety checks**: disk space, .blend changed or missing mid-job, moved render folders, single instance, keeping the computer awake.
- **Optional preflight check** for missing textures, missing linked files, unbaked simulations and more.
- **Queue files** – save a queue as a `.bbrqueue` file and switch between setups; double-click a queue file to open it.
- **Status page in any browser** (also on your phone) and a **web API** for other tools.
- **Blender add-on** “Send to Batch Renderer” – add the open file to the queue from inside Blender.
- **Rendering on several computers** – frames of an animation are shared with worker computers on the home network (Windows, macOS, Linux; any GPU), with statistics per machine and combined.
- **Keyboard shortcuts** for everything you do often.
- Runs on macOS (tested), Windows and Linux; also usable from the command line.

1.2 Guiding principles

- **The .blend file decides.** Render settings are taken from the file as saved. The app only changes what it must (output path; video output becomes an image sequence) and the few parameters you explicitly override (frame range, samples, resolution %).
- **The frame is the unit of work.** A frame is either finished and safely on disk, or it will be rendered (again). Rendering inside a frame cannot be resumed – this is a Blender limitation.

- **Unattended first.** Problems never stop the night unless they must: failing frames are skipped after their attempts, broken or changed files are set aside, and the batch carries on with the next job.

2 Installation & starting

2.1 Requirements

- Python 3.9 or newer (on the development Mac: Python 3.13 from python.org).
- One or more Blender installs (tested with Blender 5.2; written for 2.9x – 5.x).
- About 0.5 GB for the Python environment while building; the finished app is roughly 80–120 MB.

2.2 One-time setup

System	Run	What it does
macOS	double-click `setup.command`	Creates a fresh private Python environment in `.venv` and installs the packages. If macOS refuses to open it: right-click → Open.
Windows	`setup.bat`	Same, using the Python launcher.
Linux	`./setup.sh`	Same.

Installed packages: **PySide6-Essentials** (the window), **openpyxl** (Excel report), **psutil** (memory readings) and **zstandard** (reading the version of compressed .blend files).

2.3 Starting the app

Option A – a real macOS app (recommended)

Double-click `build_app.command`. It builds **Blender Batch Renderer.app** next to the script, with its own icon and name in the Dock; drag it to Applications if you like. You only need to build again when an update asks for it (2.4).

Windows: run `build_windows.bat` once. It builds the folder **Blender Batch Renderer** with `Blender Batch Renderer.exe` inside, with the same self-test.

The build leaves out the parts of Qt the app never uses (QML/Quick, PDF, networking, OpenGL, SVG, translations – see `bbr.spec`) and then starts the new app once in the background as a self-test: it draws the charts, loads a thumbnail and writes a test report. If that test fails, the script automatically rebuilds with all of Qt included and says so.

Option B – run from the source folder

Double-click `run.command` (Windows `run.bat`, Linux `./run.sh`). A Terminal window opens alongside the app; keep it open.

> **Disk space.** Once the app is built, the `.venv` folder is only needed for Option B and for rebuilding. You can delete it to save space; `setup.command` recreates it.

2.4 Updates without rebuilding

The built app consists of a **shell** – Python, Qt and the libraries, about 95 % of its size – and the **app code**. *Help → Check for updates…* asks GitHub for a newer version and installs only the app code (1–2 MB) into the app data folder (`app/`, see 10.3), next to the current version. The shell stays as it is, so the app is not rebuilt. Nothing is checked or installed unless you ask, and nothing while something renders.

Channel	Offers
Stable	Versions that were tried and approved (the `release` branch).
Alpha	Every change as soon as it is made (the `main` branch) – experimental, marked “alpha”.

- Every file of an update is checked against the checksums GitHub keeps for that version before it is installed; a damaged download is refused.
- *Use previous version* (same window) switches back. A version that does not start is replaced by the previous one automatically, with a message.
- A version that needs a newer shell (new Python, Qt or library) is not installed; the window says to build the app again (`build_app.command` / `build_windows.bat`).
- Coordinator and workers must run the same version – update both computers.
- When running from source (`run.command`), updates are not used; use `git pull` instead.

2.5 First launch

On first launch the app searches for Blender installs (macOS: `/Applications` and `~/Applications`; Windows: Program Files and Steam; Linux: PATH, `/opt`, home folder) and asks each one for its version. Add or remove installs any time under **Settings…**. Only one copy of the app can run at a time; a second copy refuses to start instead of rendering the same queue twice.

3 The window

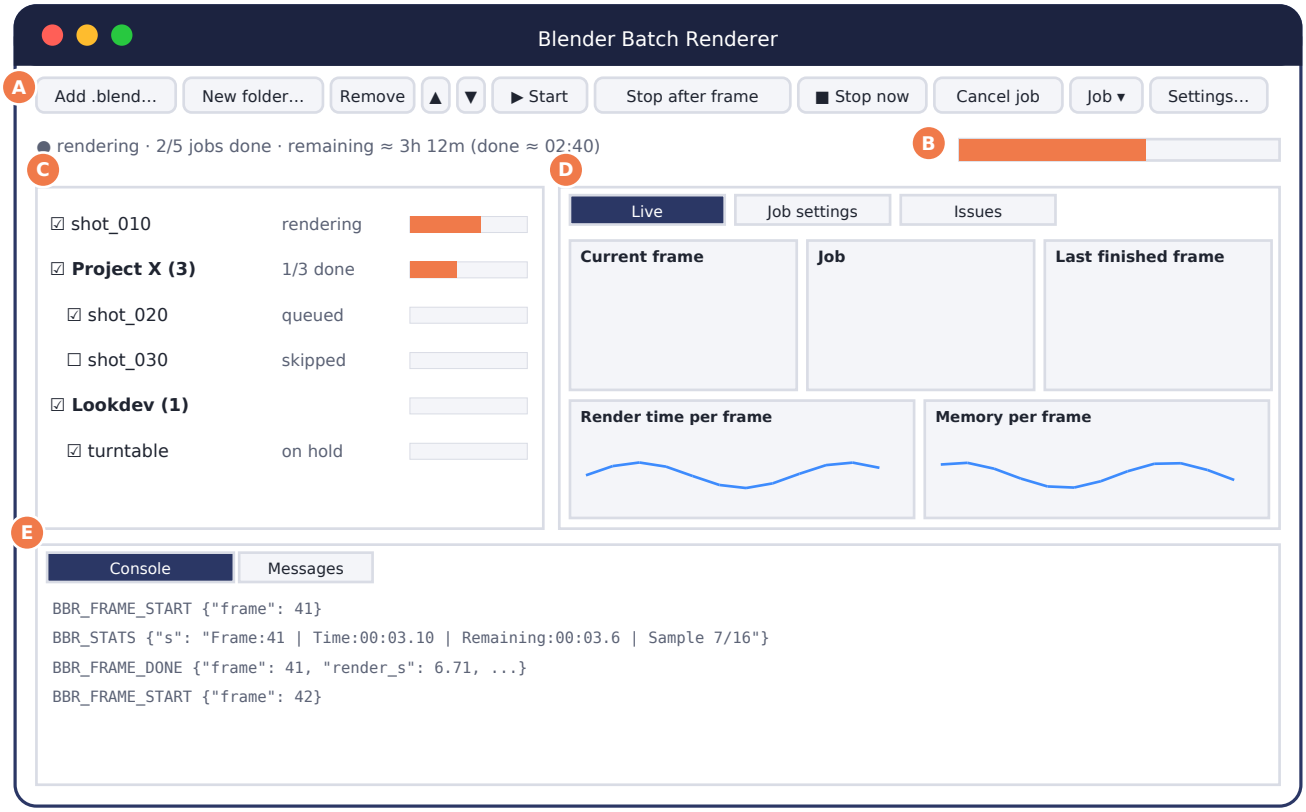


Figure 1 – Layout of the main window (schematic).

	Area	Contents
A	Toolbar & menus	Add files, folders, ordering, Start / Stop / Cancel, the **Job** ▾ menu and Settings. The menu bar has File (queue files, adding, folders, settings), Queue, Job and Help (status page, keyboard shortcuts, documentation); most commands are also in right-click menus.
B	Batch summary	State (idle / rendering / stopping), jobs done, remaining time for the whole batch with the expected finishing time, whether the computer is being kept awake, and a batch progress bar.
C	Queue	Jobs and folders in render order, each with status, progress, frames, Blender version and ETA. Checkboxes switch jobs or whole folders on/off. Drag to reorder or move between folders; drop .blend files to add them.
D	Detail tabs	**Live**, **Job settings** and **Issues** for the selected job.
E	Console & messages	Blender's console output (follows the rendering job, or shows the selected one) and a log of the app's own messages, warnings and errors.

Tip: hover the mouse over any setting, button, column header or label to see what it does.

3.1 Live tab

- **Current frame** – frame number, sample progress (e.g. *sample 64/128*), elapsed time, **Blender's own remaining-time estimate** for the frame, and memory (now / Blender's peak).

- **Job** – frames done / failed / to go, average, fastest and slowest frame, time remaining, finishing time, disk used and projected total, total render time, number of Blender runs, the Blender install used and why, engine, device, samples and resolution.
- **Last finished frame** – a thumbnail (PNG, JPG, TIFF, WebP; not EXR).
- **Charts** – render time per frame, and memory per frame (Blender's reported peak and the measured peak of the Blender process).

3.2 Job settings tab

Shows and edits the selected job's settings (see chapter 9). Changes apply the next time the job renders; finished frames are kept. Settings cannot be changed while the job is rendering.

3.3 Issues tab

Everything that went wrong or needs attention for this job: notes from Blender (e.g. video output switched to PNG), the overrides applied, preflight results, every frame that failed or is waiting for a retry with its last error, and a list of all Blender runs with start time, duration, frames rendered and result.

3.4 Machines tab and mode switch

The **Machines** tab lists the computers that render with this one, with their state and statistics, and switches sharing on or off (chapter 15). The **Mode** box at the top right of the window turns this computer into a **worker**: the queue is hidden and the window shows the worker view instead (15.1). Both are locked while something is rendering.

4 Working with the queue

4.1 Adding files

Use **Add .blend…** (⌘I), or drop .blend files onto the queue – dropped onto a folder, they go into that folder. A dialog asks for:

- **Queue folder** – where the jobs appear in the queue (or top level).
- **Render folder** – by default a folder called *render* **next to each .blend file** (`shots/shot_010.blend → shots/render/shot_010/`), so the output is always easy to find from the file. Alternatively a **custom folder**; each file then gets its own sub-folder inside it. A queue folder with its own render folder pre-selects that one.
- **Blender** – *Auto* or a specific install, and the other job settings (chapter 9). They apply to all files added in this step.

> **Re-adding.** If you add a file that was rendered into the same render folder before (and it is not in the queue), the app re-attaches to the existing job folder and keeps all finished frames. A different file with the same name gets a numbered folder (*name_2*) instead.

4.2 Folders

Folders organise the queue, e.g. one per project. Create one with **New folder…**; give it a name and optionally a default render folder, which is pre-filled whenever you add files to it. Double-click a folder (or *File → Folder settings…*) to edit it. Deleting a folder moves its jobs to the top level – no job is removed.

Move jobs by dragging them, or right-click → *Move to folder*. ▲ / ▼ move the selected job within its folder, or the selected folder among folders.

4.3 Render order and skipping

The queue renders exactly as shown, top to bottom: jobs at the top level first, then each folder's jobs in order. One job renders at a time. Untick a job or a whole folder to skip it without removing it; skipped jobs show *(skipped)* and keep their progress.

4.4 Removing jobs

Remove takes the job out of the queue only. Its folder with frames, data and logs stays on disk. Put it back later by adding the same file to the same render folder, or with *File → Add existing job folder…*.

4.5 Queue files

A queue can be saved as a file (`.bbrqueue`) and opened again later, so you can keep several setups – e.g. one per project – and switch between them. Commands in the **File** menu: **New Queue** (⌘N), **Open Queue…** (⌘O), **Open Recent**, **Save Queue** (⌘S) and **Save Queue As…** (⇧⌘S). Double-clicking a queue file in Finder opens it in the app; if the app is already running, the running app opens it.

- A queue file stores the **folders, the jobs in their order and their on/off switches** – not the progress. Progress always lives in each job's own folder, so a job shows the same progress in every queue that contains it.
- **Saving is manual.** Unsaved changes are shown by a dot in the window's close button (an asterisk in the title on other systems). Before a queue with unsaved changes is replaced (New / Open), the app asks whether to save it. The current queue is still kept when you quit, so nothing is lost if you forget to save.
- Paths are stored both absolute and **relative to the queue file**: if you move a project folder that contains the queue file and its render folders, the queue still finds its jobs.
- A queue cannot be opened or replaced while a render is running – stop the batch first.

4.6 Keyboard shortcuts

Also listed in the app under *Help → Keyboard shortcuts* (⌘/). On Windows and Linux, ⌘ is Ctrl.

Keys	Action
⌫ / **Delete**	Remove the selected job or folder (asks first; files stay on disk)
⌘⌫	Remove without asking
↩	Open the job's render folder (on a folder: folder settings)
Space	Play the preview video
⌘↑ / ⌘↓	Move the selected job or folder up / down
⌘R · ⇧⌘. · ⌘.	Start · stop after this frame · stop now
⌘N · ⌘O · ⌘S · ⇧⌘S	New · open · save · save queue as
⌘I · ⇧⌘N	Add .blend files · new folder
⌘P · ⌘, · ⌘/	Create preview · settings · list of shortcuts

The keys that act on the selected row only work while the queue list has the keyboard focus, so typing in a text field can never remove a job.

5 Rendering, stopping & resuming

Command	What happens
▶ Start (⌘R)	Renders the queue from the first job that still has work. Interrupted jobs continue with their first missing frame.
Stop after frame	The current frame finishes and is saved, then the batch stops.
■ Stop now (⌘.)	Blender is ended immediately. The interrupted frame does not count as a failure and is rendered again next time.
Cancel job	Ends the current job, marks it *cancelled* and continues with the next job. Finished frames are kept.
Quit (close window)	If a render is running, asks: *Stop now and quit*, *Finish this frame, then quit*, or *Keep rendering*. Either way, the next Start continues where it stopped.
Retry failed frames	Queues a job again and renders only the frames that gave up (also for failed, cancelled and on-hold jobs).
Continue with changed file	For a job on hold because its .blend changed: keep finished frames, render the rest from the changed file.
Start job over…	Forgets the job's progress and renders every frame again (existing files are overwritten).
Create preview (⌘P)	Builds the MP4 preview from the frames finished so far.

5.1 How resuming works

After every finished frame the app writes the job's complete state to `job.json` in the job folder (written to a temporary file and renamed, so it is never half-written). When a job starts, the app checks that every frame recorded as finished still exists on disk and is not empty; missing frames are simply rendered again. Blender is then told which frames to skip.

Inside Blender, each frame is saved under a temporary name (`.tmp_…`) and renamed only when it is complete. If Blender is killed while saving, only the temporary file is left, and it is cleaned up before the next run.

> **Crashes and power loss.** If the app itself crashes or the computer loses power, nothing special is needed: on the next launch a job that was *rendering* is shown as *stopped (interrupted)*. With *Resume an interrupted batch automatically* enabled in Settings, the batch restarts by itself.

5.2 Job states

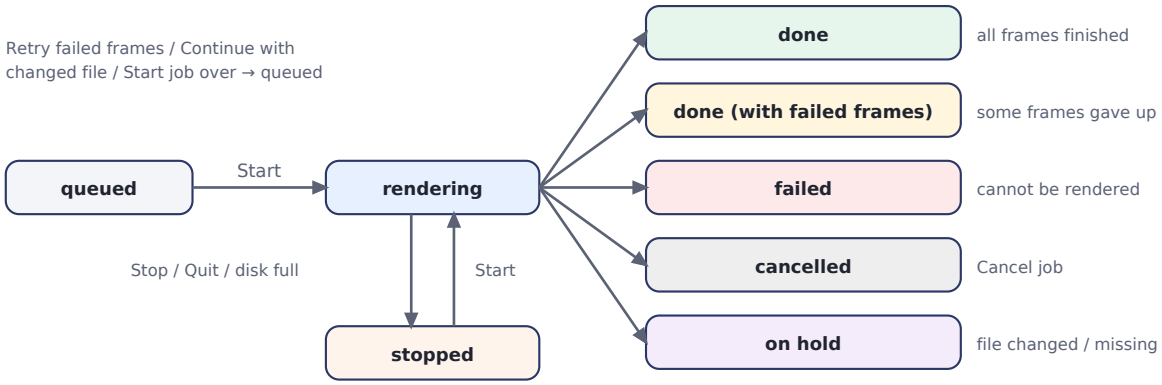


Figure 2 – Job states.

State	Meaning	Rendered by Start?
queued	Waiting for its turn.	yes
rendering	Blender is working on it (or loading, checking, waiting to retry, making the preview).	–
stopped	Stopped by you, by quitting, by a full disk, or interrupted by a crash.	yes – continues
done	All frames finished.	no
done (with failed frames)	Finished, but some frames gave up after all attempts.	no – use Retry
failed	The file could not be rendered at all (e.g. no camera, cannot load).	no – use Retry
cancelled	Cancelled by you.	no – use Retry
on hold	Needs a decision: the .blend changed since frames were rendered, or it is missing.	no – see 7.2

6 Failures, retries & restarts

6.1 Frame failures

If Blender crashes or reports an error while rendering a frame, that frame's attempt counter goes up, the console log is copied to `logs/crashes/` together with Blender's own crash report, and Blender is started again after **Retry delay**. After **Attempts** failed attempts the frame is marked *failed* and the job continues with the next frame. Failed frames are listed in the Issues tab and in the reports.

6.2 Files that will not render

If Blender fails before rendering any frame (e.g. the file cannot be read), this counts as a failed start; after the same number of attempts the job is marked *failed* and the batch moves on. Some problems are recognised immediately and fail the job without retries – for example a scene without an active camera.

6.3 Frame timeout

With a **Frame timeout** set, a frame that takes longer is ended and counted as a failed attempt – useful against renders that hang. Leave it at *no limit* unless you know your frame times.

6.4 Restart Blender

Normally one Blender process renders all remaining frames of a job, which saves loading time. Over a long sequence Blender's memory use can grow until it crashes. With **Restart Blender** set to N frames, a fresh Blender is started after every N frames. These planned restarts are never counted as failures; the cost is one file load per restart.

7 Safety checks

7.1 Disk space

Before each Blender run and after every frame, the app checks the free space on the render drive. If it is below the limit in Settings (default 5 GB) – or below three average frames, whichever is more – the batch stops right after the current frame. If Blender reports *No space left on device* while saving, the same happens. No frame is marked as failed; free some space and press Start.

7.2 Changed or missing .blend files

The app remembers the modification time and size of each .blend when its frames are rendered. If the file was saved again after some frames were finished, continuing could mix two versions of the scene in one sequence. The job is therefore put **on hold** and the batch continues with other jobs. Decide in the **Job** ▾ menu: *Continue with changed file* (keep finished frames) or *Start job over*. A .blend that no longer exists also puts its job on hold instead of failing it.

7.3 Moved render folders

Frames are recorded relative to their job folder, so moving or renaming the render folder – or the drive getting a different name – does not lose any frames. A moved job folder is reported as missing when the app starts; put it back with *File → Add existing job folder…*.

7.4 One copy at a time

An operating-system file lock ensures that only one copy of the app (window or command line) works on the queue. The lock is released automatically if the app crashes, so it never needs cleaning up.

7.5 Keeping the computer awake

While a batch runs, the app prevents the computer from sleeping (macOS: `caffeinate`; Windows: system execution state; Linux: `systemd-inhibit`).

> **Limits.** On macOS this only works while the Mac is plugged in, and a MacBook still goes to sleep when its lid is closed without an external display.

8 Blender versions

Every .blend file records the Blender version it was saved with. The app reads it directly from the file header – also for compressed files – without starting Blender, and picks an install:

- an install with the **same major.minor version** (the newest patch release if there are several);
- otherwise the **next newer** version (the lowest version above the file's, again its newest patch);
- otherwise – the file is newer than every install – the newest install, with a warning in the job.

The choice and the reason are shown in the queue, the Live tab and the report. In the job settings you can force a specific install instead of *Auto*. Several versions can be mixed freely in one batch, since every job starts its own Blender.

Preferences & add-ons. Blender is started with your normal user preferences, so add-ons and GPU settings are the same as in your interactive Blender. Drivers that use Python expressions only work if *Auto Run Python Scripts* is enabled in Blender's preferences – the same as for any command-line render.

9 Settings reference

9.1 Job settings

Set when adding files, edited later in the **Job settings** tab. Defaults for new jobs are set in **Settings…**.

Setting	Default	Description
Render folder	next to the .blend	A *render* folder beside each .blend, or a custom parent folder.
Blender	Auto	Which install renders the job (chapter 8).
Start frame / End frame	from file	Override the frame range. Unticked = use the file's range.
Samples	from file	Override render samples (Cycles samples or EEVEE render samples).
Resolution	from file	Override the resolution percentage.
Attempts	3	How often a frame (or loading the file) is tried before giving up.
Retry delay	30 s	Pause before Blender is started again after a failure.
Frame timeout	no limit	End a frame that takes longer than this; counts as a failed attempt.
Restart Blender	never	Fresh Blender process every N frames (6.4).
MP4 preview	on	Build the preview video when the job finishes.
Check file first	off	Run the preflight check (chapter 12) before the first render of the job.

Setting	Default	Description
Ignore add-ons	off	Workers render the job even without the add-ons it seems to use (15.2).
File transfer	Smart	Smart: files once, frame files when needed; Full copy: everything before the first frame (15.3).
Test frame	on	Test frame before a computer joins (15.4).
Devices	as saved in the file	For each computer: GPU, CPU, or Off (never renders this job). Cycles only (15.2).

9.2 App settings

Setting	Default	Description
Blender installs	found automatically	List of installs with their versions; Add…, Remove, Scan.
Keep awake while rendering	on	Prevent sleep while a batch runs.
Resume on start	off	Restart an interrupted batch when the app launches.
Console lines	5000	How much Blender output the console keeps in memory (the log files always contain everything).
Min. free space	5 GB	Disk-space limit for 7.1.
Status page and API	on	Chapter 13.
Port	8642	Network port of the status page and API.
Home network access	off	Makes the status page reachable from phones and other computers at home; they need the key.
Access key	random	Part of the link for other devices; *New key* locks out old links.
Mode (top right)	Coordinator	Coordinator keeps the queue; Worker renders for a coordinator (15).
Share jobs with workers	off	Machines tab: other computers may render frames of the queue.
This computer renders too	on	Machines tab: off = the coordinator only hands out frames.
Pairing code	random	Machines tab: 6 digits a new worker enters once; *New code* for new workers.
Worker: keep downloaded jobs up to	50 GB	Worker window: size of the cache of job copies (15.8).
Worker: connect automatically when the app starts	off	Otherwise press *Connect* in the worker window (15.1).
Worker: folder for downloaded files	app data folder	Worker window: *Folder…* (15.8).

9.3 What the app changes in your file

Only in the background Blender session – **the .blend file itself is never saved or modified**:

- the output path (frames go to the job's `frames/` folder);
- video output (FFmpeg etc.) becomes a PNG sequence, keeping 8- or 16-bit colour depth;
- compositor *File Output* nodes write into `frames/<node name>/`;
- the overrides you set (frame range, samples, resolution %).

10 Output folders & data files

```
<render folder>/<file name>/
  frames/            <name>_0001.png …      finished frames only
  frames/<node>/     output of compositor File Output nodes
  preview/           <name>.mp4             preview video
  data/              frames.csv             one row per frame event, written live
                     attempts.csv           one row per Blender run
                     report.xlsx            summary, frames, attempts + charts
                     preflight.json         result of the optional file check
  logs/              attempt_001.log …      complete Blender console of every run
                     preflight.log, preview.log
  logs/crashes/      logs + Blender crash reports of failed runs
  logs/tmp/          Blender's temporary folder for this job
  job.json           the job's full state – this is what makes resuming work
```

10.1 frames.csv

Column	Meaning
frame, status	Frame number; *done*, *pending* (will be retried) or *failed*.
attempt_no, attempts_for_frame	Number of the Blender run; how many times this frame was tried.
finished_at	Date and time.
render_s, save_s	Seconds for rendering and for saving the frame.
blender_peak_mb	Highest memory value Blender reported during the frame.
process_peak_mb	Highest measured memory of the Blender process during the frame.
samples, size_bytes, file	Samples, file size and path of the frame.
error	Last error for frames that failed.

10.2 report.xlsx

Rewritten whenever a job finishes, stops or is cancelled. **Summary**: file, versions, engine, device, resolution, samples, frame range, fps, output format, totals (frames done/failed, Blender runs, total render time, wall-clock time, average/fastest/slowest frame, memory peaks, disk used, preview path) with charts of render time, memory and file size per frame. **Frames**: one row per frame. **Attempts**: one row per Blender run with duration, frames rendered, exit code, result and log file. If the report is open in Excel and locked, a timestamped copy is written instead.

10.3 Where the app keeps its own data

macOS: `~/Library/Application Support/BlenderBatchRenderer/`; Windows: `%APPDATA%\BlenderBatchRenderer`; Linux: `~/.local/share/BlenderBatchRenderer`. It contains `settings.json` (installs and defaults), `queue.json` (folders and the order of job folders) and `app.lock`. On a worker it also holds `worker_cache/` (downloaded job copies) and `worker_logs/` (15.8). Everything about a job lives in its own job folder.

11 Preview video

When a job finishes (if *MP4 preview* is on), or when you choose **Create preview**, the finished frames are loaded into Blender's video editor in a second, add-on-free Blender run and encoded as H.264 MP4 at the scene's frame rate and output resolution (rounded to even pixels). The result is `preview/<name>.mp4`; double-click a job's folder or right-click → *Play preview*.

- Frames that failed appear as short black gaps, so timing stays correct.
- PNG/JPG sequences are shown as they are; EXR sequences use the scene's view transform and look.
- The preview is meant for checking, not as a final deliverable – use your editor (e.g. DaVinci Resolve) with the image sequence for that.

12 Preflight check

With *Check file first* enabled, Blender opens the file once before the first render and reports problems, which appear as warnings and in the Issues tab (and in `data/preflight.json`). The job still renders – the check informs, it does not block. It looks for:

- a scene without an active camera;
- missing images, linked libraries, sounds, movie clips, fonts, volumes and cache files;
- simulation caches that are not baked (cloth, soft body, particles and hair dynamics, dynamic paint, rigid body world) and fluid domains using a replay cache – these can come out differently when a job resumes mid-sequence;
- Cycles set to GPU without any GPU enabled in Blender's preferences;
- notes: video output that will become PNG, File Output nodes that will be redirected.

13 Status page & API

While the app is open it runs a small web server. In a browser it shows a **status page**; for other programs it offers an **API**. Open the page with *Help* → *Status page* or at `http://127.0.0.1:8642/`.

13.1 The status page

- Batch state, jobs done, remaining time for the batch and the expected finishing time.
- The frame being rendered: sample progress, Blender's remaining time, the job's remaining time, memory, and a thumbnail of the last finished frame.
- The queue with its folders: status, notes, progress, frames and remaining time per job; a *retry* link for failed, cancelled and on-hold jobs.
- Buttons for **Start**, **Stop after frame** and **Stop now**, and Blender's console (fold-out).

- It refreshes every two seconds and adapts to phone screens and dark mode.

13.2 On your phone or another computer

In *Settings → Status page & API*, tick **Home network access**. *Copy link* copies a link such as `http://192.168.1.20:8642/?key=…`; open it on the phone (and add it to the home screen). Devices without the key are refused.

> **Outside your home.** Away from home the app is not reachable – on purpose. If you want that, use a VPN such as Tailscale (your phone then behaves as if it were at home). Do not open a port on your router.

13.3 Security

- By default the server only accepts connections from the computer itself.
- Other devices (if allowed) must send the key. *New key* in Settings makes all old links stop working.
- Requests that change something must carry a special header, which ordinary web pages cannot send to the app; requests with an unexpected host name are refused. A random website you visit therefore cannot control the app.
- The API only works with files on this computer and never deletes files: removing a job only takes it out of the queue.

13.4 API reference (version 1)

All answers are JSON. From the same computer, requests that change something need the header `X-BBR-Request: 1`; other devices send `X-API-Key: <key>`.

Request	Purpose
GET /api/v1/status	Batch state, live frame information, estimates, current queue file.
GET /api/v1/queue	Folders and jobs in render order (status, progress, frames, ETA, disk use).
GET /api/v1/jobs/<id>	Job details: settings, scene facts, preflight, problem frames, Blender runs, frame times.
GET /api/v1/jobs/<id>/thumbnail	The last finished frame as an image (not for EXR).
GET /api/v1/console?since=N	Blender console lines after line number N (optional &job=<id>).
POST /api/v1/jobs	Add a job. Body: `blend` (path, required), optional `folder` (queue folder name or id, created if missing), `render_root` ("" = next to the .blend), `frame_start, frame_end, samples, resolution_pct, max_attempts, retry_delay_s, frame_timeout_min, restart_every, make_preview, preflight, blender, start`.
POST /api/v1/batch/start	Start the batch.
POST /api/v1/batch/stop	Body `{"mode": "after_frame"}` or `{"mode": "now"}`.
POST /api/v1/batch/cancel-job	Cancel the job being rendered and continue with the next.
POST /api/v1/jobs/<id>/retry · /reset · /preview	Retry failed frames · start over · create the preview.

Request	Purpose
POST /api/v1/jobs/<id>/enabled	Body `{"enabled": false}` – switch a job off or on.
DELETE /api/v1/jobs/<id>	Remove the job from the queue (files stay on disk).
POST /api/v1/queue/open	Body `{"path": "…/x.bbrqueue"}` – open a queue file in the window.

Errors come back with an HTTP error code and `{"error": "…"}`, e.g. 400 for a missing .blend, 404 for an unknown job, 409 when a job is rendering.

```
curl http://127.0.0.1:8642/api/v1/status
curl -X POST -H "X-BBR-Request: 1" \
     -d '{"blend": "/Users/me/shots/shot_010.blend", "folder": "Project X", "start": true}' \
     http://127.0.0.1:8642/api/v1/jobs
```

14 Blender add-on “Send to Batch Renderer”

The add-on adds the file open in Blender to the queue with one click. It uses the API, so the app must be running with the status page & API switched on (the default).

14.1 Installing

- In Blender: *Edit → Preferences → Add-ons*, open the ▾ menu, choose *Install from Disk…* and pick `blender_addon/bbr_send_to_queue.py` from the app's folder. Tick it to enable it.
- Install it in each Blender version you use. If you changed the app's port, enter the same port in the add-on's preferences.

14.2 Using it

Render → Send to Batch Renderer…, or the **Batch Renderer** panel under *Properties → Output*. A small dialog asks for:

- the render folder – next to the .blend (default) or a custom folder;
- an optional queue folder (created if it does not exist);
- an optional frame range (pre-filled with the scene's range);
- whether to save unsaved changes first (on by default – the batch renders the file as it is on disk);
- whether to start rendering right away.

Open status page in the same panel opens the status page in the browser. If the app is not running, the add-on says so instead of failing silently.

15 Rendering on several computers

Several computers can render the frames of the same animation together. One of them – the **coordinator** – has the queue as usual. The others run the same app in **worker** mode: they ask the coordinator for frames, render them and send them back. Finished frames, reports and the preview end up in the coordinator's job folders exactly as if it had rendered everything itself. No shared folder or NAS is needed: the app transfers everything itself.

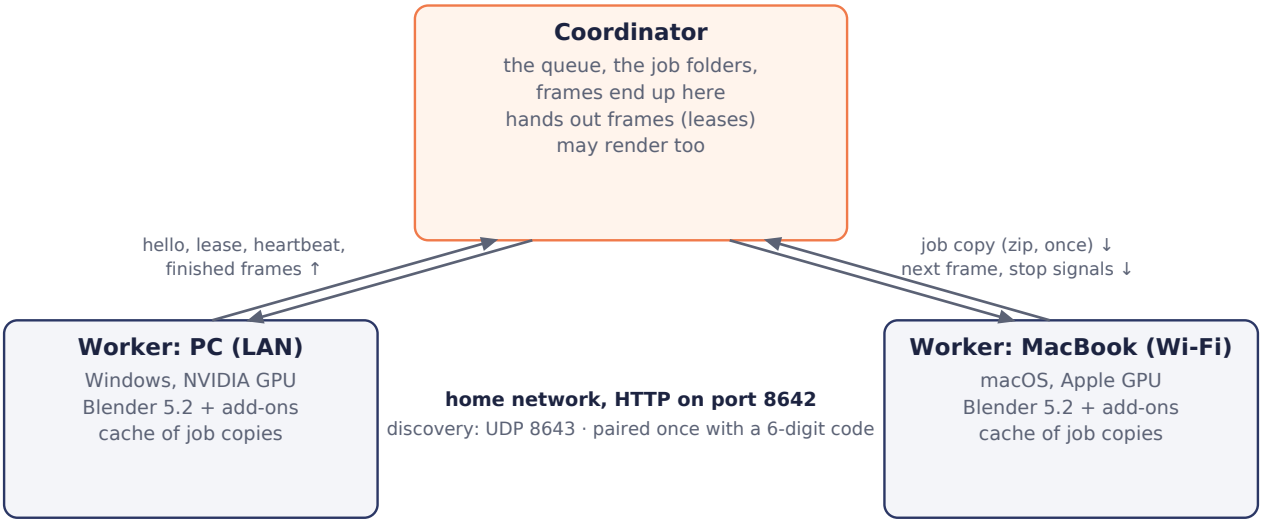


Figure 3 – A coordinator with two workers.

15.1 Setting it up

- **On every computer**: install the app and the Blender version(s) your files need, plus the add-ons your scenes use (for example LensSim). Add-ons are not copied between machines – each one needs its own installation (and licence). Enable them in Blender's preferences as usual.
- **On the coordinator**: open the **Machines** tab and tick *Share jobs with workers*. The tab shows a 6-digit **pairing code** and the address workers connect to. When the firewall asks, allow incoming connections for the app.
- **On each worker**: switch the mode (top right of the window) to **Worker**. Press *Find on network* (or type the coordinator's address), enter the pairing code and press *Pair*. This is needed only once. Afterwards the worker connects only when you press **Connect** in its window – or by itself when the app starts, if you tick *Connect on start*. **Disconnect** stops it (not while it is rendering: pause or stop first).
- Press **Start** on the coordinator as usual. Workers that are online and switched on join in.

Locked while rendering. The mode switch, sharing and the worker's settings are **locked while something is rendering**. On a worker, use *Pause after frame* or *Stop now* instead.

15.2 What happens

- **The job's files.** Before the first worker renders a job, the coordinator lists every file the job needs and writes copies of the .blend and of every **linked library** (also libraries linked by libraries) whose paths point to those files. Your original files are not changed. The files are sent one by one, each with a checksum (section 15.3).
- **Frames, not ranges.** Workers ask for one frame at a time and keep Blender open between frames, so a fast PC simply renders more frames than a laptop. Each frame is *leased* to one machine; while a worker sends its heartbeat every few seconds, the lease stays valid.
- **Delivery.** A finished frame is sent to the coordinator right away (in the background – Blender already renders the next one) and saved there via a temporary name, like local frames. The worker deletes its copy once the coordinator confirmed it.
- **Add-ons and versions.** The coordinator detects which add-ons a file uses (from its node groups, custom properties and render handlers) and gives frames only to workers with the same Blender version (major.minor) and those add-ons enabled. The Machines tab shows why a worker gets no frames. If the add-ons do not change the image, tick *Ignore add-ons* in the job settings.
- **Render engines.** Cycles, EEVEE and Workbench are supported. Cycles uses each worker's own device preferences (OptiX/CUDA on NVIDIA, Metal on Apple Silicon); the device shows in the Machines tab.
- **Devices.** In a job's settings, choose for each computer *From file*, *GPU*, *CPU* or *Off*. For example, a scene using an add-on that only works on NVIDIA GPUs renders on the PC's GPU and on the Mac's CPU (or is switched off there); a scene that needs more VRAM than the PC has renders on the Mac's GPU and is switched off for the PC. *GPU* uses the GPU set in that computer's Blender preferences, or the first one found.

15.3 Sending the files

Group	Examples	Sent
Scene	the .blend and its linked libraries	before the first frame; only changed files when you save again
Assets	textures, HDRIs, UDIM tiles, Alembic/USD, single VDBs, movies, fonts	once; kept on the worker by checksum and reused by every job that uses them
Frame files	image sequences, VDB sequences, simulation and fluid caches on disk	**Smart**: only the files a frame needs, just before it renders (the next frame's files load while the current one renders), deleted afterwards. **Full copy**: all before the first frame.

- **File transfer** (job setting): *Smart* (default) or *Full copy*. Only frame files the job's frame range actually uses are sent at all. For caches, the neighbouring frames are included, because motion blur reads them.
- **Automatic fallback.** After opening the job, the worker's Blender checks every file the scene refers to. If something is missing, or Blender reports that it cannot read a file while rendering a frame, the worker switches that job to Full copy and renders the frame again. Such a frame is never delivered or counted as failed, and the coordinator shows a message.
- **Checksums.** Every file is checked after transfer, in both directions. A damaged download is fetched again (three times at most); a damaged frame is sent again. The coordinator remembers checksums and reads a file again only when its size or date changes. A file that

changes during the batch makes the coordinator renew the job's file list.

- **Compression** only where it helps: a file that does not shrink by at least 10 % (EXR, PNG, VDB, Alembic …) is sent as it is.
- **Version check.** Coordinator and workers must run the same version of the app; another version is refused with a clear message.
- **Unbaked simulations** can come out differently on another computer – such jobs stay on the coordinator until they are baked.

15.4 Consistency check and the last frames

With *Test frame* (job setting, on by default), a computer that joins a job first renders a frame that is already finished. The coordinator compares the two images in Blender, block by block, so render noise – which differs between CPU, GPU and GPU makers – averages out, while a missing add-on effect, other colours or missing textures stand out. A computer whose frame differs is left out of that job; its test frame is kept in `data/consistency/`. Untick the setting to use it anyway. The check runs again when the job's files or the computer's device setting change.

Is it worth it? A computer gets no more frames of a job when the others would finish all remaining frames before it finishes one (its render time plus moving that frame's files). So a slow laptop does not hold up the end of a job.

15.5 Stopping, failures and lost machines

Situation	What happens
Pause / stop one worker	*Pause after frame*: it delivers its current frame, then takes no more. *Stop now*: it drops its frame at once. *Resume*: it takes frames again. On the worker itself, in the coordinator's Machines tab (buttons or right-click) and on the status page. Its work goes to the other computers; nothing counts as failed.
Stop after frame	Every machine finishes and delivers the frame it is working on, then the batch stops.
Stop now / Cancel job	Every machine drops its frame at once; nothing is counted as failed.
Pausing one computer	**Pause after frame** (it delivers its frame, then takes no more), **Stop now** (it drops its frame, which goes to another computer; nothing counts as failed) and **Resume** – in the worker window, in the Machines tab (select or right-click the computer) or on the status page. The others carry on with the job.
A worker crashes on a frame	The frame counts one failed attempt (with the machine's name) and goes preferably to another machine. A worker that keeps crashing on a job or cannot open it is taken off that job for the rest of the batch.
A worker disappears (sleep, Wi-Fi drop, quit)	After 45 seconds without a heartbeat its frame goes back into the pool. A frame it finishes later is still accepted if nobody delivered it meanwhile.
The coordinator quits	Workers finish their frame and keep it until the coordinator is back, then deliver it.
The .blend or a library is saved again	A new file list is made; workers fetch only the changed files. The usual on-hold rule (7.2) applies.

15.6 Statistics

- **Machines tab** (coordinator): per machine its state, what it is rendering (frame, samples, Blender's remaining time), device, Blender versions and add-ons, frames this batch, average frame time, failed attempts, data with the measured transfer speed, and all-time totals; above the table the combined figures and the speed of all machines together (frames per hour). Untick a machine to leave it out (when no batch is running).
- **Live tab**: frames per machine for the selected job, which frames are being rendered where, and the size of the job copy. Time remaining uses the combined speed of all machines that can render the job.
- **Reports**: `frames.csv` has a *machine* column; `report.xlsx` gets a **Machines** sheet (frames, share, render time, average / fastest / slowest frame, failed attempts, disk) with charts.
- **Worker window**: this session and all time, per job; the frame being rendered with samples, remaining time, memory and device; downloads; the Blender console.
- **Status page**: a Machines section with the same per-machine and combined figures.

15.7 Security

Sharing makes the app reachable on the home network. Every worker gets **its own key** when it pairs; the coordinator stores only a fingerprint of it. A key works only together with that worker's machine id. Each pairing code works **once** and changes after a successful pairing; it only works while sharing is on, and repeated wrong codes are blocked. *Unpair and remove* (right-click a machine in the Machines tab) revokes that worker's key at once – it has to pair again with the current code to come back. Without a valid key, nothing can be downloaded or uploaded. The status page still needs its own key for other devices. Use the app only on networks you trust; for working away from home use a VPN such as Tailscale.

15.8 Where the files are

What	Where
Job files on the coordinator	`<job folder>/package/v_<version>/` – the .blend copies and the file list; renewed when the scene changes.
Files on a worker	`worker_cache/` in the app data folder (10.3), or the folder chosen with *Folder…*: `store/` (files by checksum), `jobs/` (each job's layout, linked to the store). The longest unused files are removed when the size limit is reached; *Clear* empties it. On Windows, choose a short folder (e.g. D:\bbr) if file names are long.
Frames on their way	`worker_cache/work/<job>/` – deleted once the coordinator has confirmed each frame.
Worker crash logs	`worker_logs/` in the app data folder.
Known workers	`settings.json` on the coordinator – listed (offline) after a restart until they connect again.

16 How it works

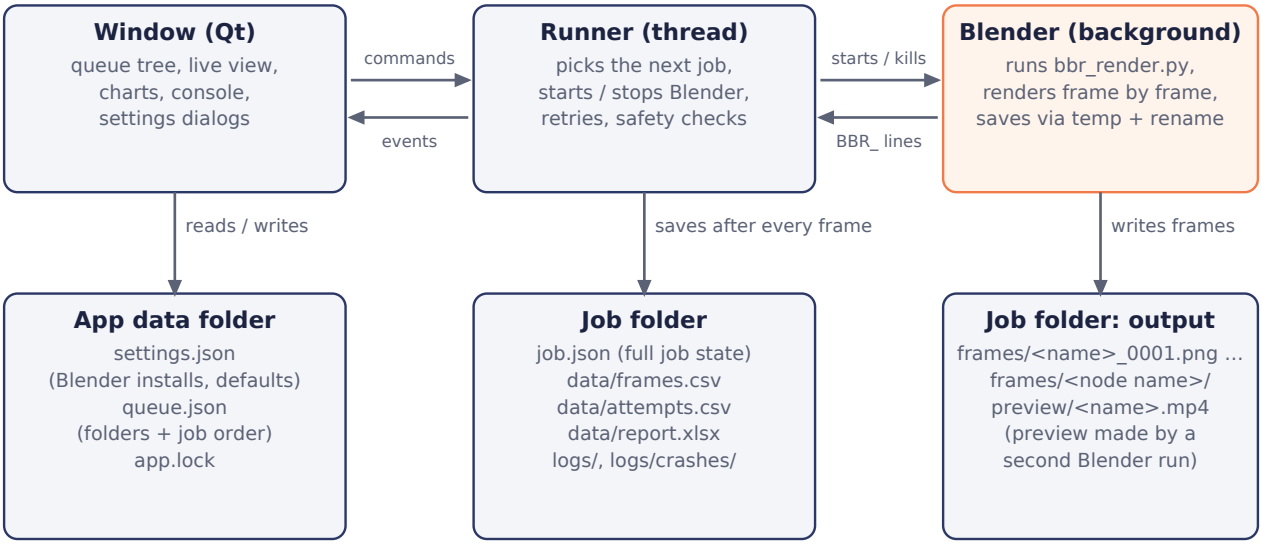


Figure 4 – Components and data flow.

The app has three layers. The **window** (Qt) shows the queue and live data and sends commands. The **runner** runs in a background thread: it picks the next runnable job, performs the safety checks, starts Blender as a separate process, reads its output line by line, samples its memory, handles stops, timeouts and retries, and saves state. **Blender** runs headless with a small script (`bbr_render.py`) that applies the overrides, renders frame by frame and reports back.

16.1 Communication with Blender

The render script prints one line per event, starting with BBR_ followed by JSON. Everything else Blender prints is kept as console output.

Line	Meaning
BBR_JOB_INFO	Scene facts after overrides: engine, device, frame range, fps, resolution, samples, output format, colour management, File Output nodes, notes.
BBR_PLAN	Total frames and frames still to render.
BBR_FRAME_START	A frame begins.
BBR_STATS	Blender's render statistics (samples, time, remaining time, memory), at most four times per second. Needed because Blender 5 no longer prints progress lines in background mode.
BBR_FRAME_DONE	A frame is safely on disk: file, size, render and save time, peak memory, samples.
BBR_FRAME_ERROR / BBR_FATAL	A frame failed / the job cannot be rendered at all.
BBR_CHUNK_DONE / BBR_ALL_DONE	Planned restart point reached / every frame finished.
BBR_WARN	A note for the user (e.g. output switched to PNG).

Line	Meaning
BBR_NEED_FRAME	Blender asks which frame to render next; the app answers on Blender's input with `FRAME n` or `DONE`. This is how frames are shared between machines while Blender stays open.
BBR_PACKAGE / BBR_ADDONS	The job's file list / list of enabled add-ons.
BBR_INPUT_MISSING	A worker's Blender misses files the scene needs (triggers Full copy).
BBR_COMPARE	Result of comparing two renders (consistency check).

Blender versions up to 4.x also print their own status lines (`Fra:… | Time:… | Remaining:… | Sample …`); these are read as well.

16.2 Stopping cleanly

Blender runs in its own process group. *Stop now* sends it a terminate signal and, if it has not ended after five seconds, kills it. *Stop after frame* waits for the next `BBR_FRAME_DONE` and then ends Blender while it is between frames. A frame interrupted by the user is never counted as a failed attempt.

16.3 Estimates

- **Job time remaining** = Blender's remaining time for the current frame + (frames left − 1) × the average of the last ten frames.
- **Batch time remaining** = the sum over all jobs that will still render. Jobs that have never started have no frame times yet and are counted separately (shown as “no estimate”). Loading time per Blender start is not included.
- **Projected disk use** = average frame size × number of frames.

16.4 Source code layout

File	Purpose
bbr/models.py	Job, job settings and app settings; frame bookkeeping and estimates.
bbr/jobqueue.py	Queue order, folders, adding / re-attaching jobs.
bbr/runner.py	The batch engine: job loop, Blender runs, retries, safety checks, preview.
bbr/process.py	Starting, reading, memory sampling and killing a Blender process.
bbr/blender_scripts/	Scripts that run inside Blender: render, preflight, preview, shared helpers.
bbr/parse.py	Reading BBR_ lines and Blender's statistics; formatting of times and sizes.
bbr/installs.py, blendfile.py	Finding Blender installs; reading a .blend's version.
bbr/reports.py, preview.py	CSV and Excel output; building the MP4 preview.
bbr/storage.py, keepawake.py	Safe file writing, app data folder, instance lock, free space; sleep prevention.
bbr/gui/	The window: main window, queue tree, dialogs, charts, app start-up.
bbr/cli.py	Command-line interface.
bbr/api.py, bbr/control.py	Web server, status page, API routes and security; what the API can read and do.
bbr/assets/status.html	The status page (plain HTML and JavaScript).
bbr/gui/api_bridge.py	Runs API requests on the window's thread.
bbr/scheduler.py	Leases: which machine renders which frame; machines and their statistics.
bbr/workerapi.py, bbr/worker.py	Coordinator endpoints for workers and discovery; the worker client.
bbr/package.py	The job's file list and frame mapping (with blender_scripts/bbr_package.py).
bbr/hashing.py	Checksums, remembered until a file changes.
blender_scripts/bbr_compare.py	Compares two renders for the consistency check.
bbr/gui/machines_view.py, worker_view.py	Machines tab; the window in worker mode.
blender_addon/	The “Send to Batch Renderer” add-on for Blender.
bbr.spec, build_app.command	Building the trimmed macOS app.
tests/	Engine, API, queue-file and add-on tests with a simulated Blender.
tools/make_icon.py, docs/make_docs.py	Draws the app icon; builds this document.

17 Command line

Everything except the window is also available in a terminal, from the app's source folder:

```
.venv/bin/python -m bbr scan                       # find Blender installs and save them
.venv/bin/python -m bbr scan "/Applications/Blender 4.app"   # … plus extra installs
.venv/bin/python -m bbr add shot.blend --root /path/to/render --folder "Project X" \
        --start 1 --end 100 --samples 64 --pct 50 --attempts 3 --delay 30 --timeout 20 \
        --blender auto --no-preview --preflight
.venv/bin/python -m bbr list                       # queue with status and progress
.venv/bin/python -m bbr run                        # render the queue; Ctrl+C = stop now
.venv/bin/python -m bbr worker --code 123456       # be a worker (finds the coordinator, pairs once)
.venv/bin/python -m bbr worker --server 192.168.1.20 --name "Render PC"
```

The command line and the window share the same queue, settings and lock – only one of them can render at a time. `run` also starts the status page and API.

18 Troubleshooting & limitations

Symptom	What to do
A job is *on hold*	Its .blend changed or is missing – see the status note and section 7.2.
The batch stopped with “disk almost full”	Free space on the render drive (or lower the limit in Settings) and press Start.
Frames keep failing	Open *Issues* for the error, and the log in `logs/crashes/`. Memory problems: try *Restart Blender* or lower samples / resolution.
Wrong Blender version used	Check the reason in the Live tab; add the missing install under Settings or choose one in the job settings.
“Job folder missing” on start	The render folder was moved or renamed: *File → Add existing job folder…*.
No sample progress in the Live tab	Look for BBR_STATS lines in the console; their format can change between Blender versions.
“Already running” message	Another copy of the app (or the command line) is open.
Mac went to sleep	Keep it plugged in; a closed MacBook lid without an external display still sleeps.
Status page does not open	Check *Settings → Status page & API*; the Messages tab says if the port is taken – choose another port (and enter it in the Blender add-on too).
Phone cannot reach the page	Tick *Home network access*, use the copied link (it contains the key), make sure the phone is on the same Wi-Fi, and allow the app in the Mac's firewall.
Add-on: “not running”	Start the app; check that the port in the add-on's preferences matches the app.
A worker gets no frames	The Machines tab says why: other Blender version, missing add-on, switched off, or the job copy could not be made (Messages tab).
Worker cannot find the coordinator	Both on the same network? Sharing on? Type the address shown in the Machines tab; allow the app in the firewall (Windows asks on first use).
Frames from a worker look different	Compare Blender version, add-on versions and colour management; render a test frame on each machine.

Known limitations

- A frame interrupted mid-render starts again from zero (Blender cannot resume inside a frame).
- Compositor *File Output* files are written directly by Blender, not via temporary names, and are not checked on resume.
- Stereo / multi-view renders: only the main view is saved by the render script (untested).
- Multilayer EXR output is supported in principle but has not been tested yet.
- Thumbnails are not available for EXR frames.
- Unbaked simulations may differ after a resume – bake them first.
- Each Blender start loads your add-ons; slow add-ons add time to every start (and every planned restart).
- Only the active scene of a file is rendered; put view layers you want separately into separate files.
- Several computers: unbaked simulations are not shared; geometry-nodes bakes and image sequences used in unusual places (e.g. a geometry-nodes Image Texture) are sent completely instead of frame by frame. Third-party render engines are not supported for sharing yet. Add-ons are detected by heuristics; a scene that uses an add-on only through a driver or script may not be recognised.

Testing

`python3 tests/test_api.py` tests the API and its security, queue files, the default render folder and the Blender add-on. `python3 tests/test_engine.py` runs 17 scenarios against a simulated Blender: crashes and retries, frames that always fail, stop and resume, quitting mid-render, files that do not load, missing camera, video-to-PNG switching and overrides, cancelling, re-attaching, Blender-5-style output, planned restarts, full disk, free-space limit, changed and missing .blend files, moved render folders, folders and the single-instance lock. `python3 tests/test_distributed.py` simulates a coordinator with workers: job copies with nested linked libraries and add-on detection, pairing and access checks, discovery, faster and slower workers, a missing add-on, crashing workers, a worker that disappears, stop now / stop after frame, and the coordinator rendering together with a worker. `python3 tests/test_updater.py` tests updates against a local stand-in for GitHub. `python3 tests/test_transfer.py` tests the file transfer: Smart transfer and reuse across jobs, damaged downloads and uploads, the switch to Full copy, the version check, the consistency check, the last-frames rule and remembered checksums.